Лабораторная работа № 2. Модели. CRUD.

Цель работы: разработка моделей, научиться проводить миграцию данных, работа с базой данных.

Теоретическое обоснование

1. Модели в Django

Модель (Model) является ключевым компонентом архитектуры MTV (Model-Template-View). Она представляет собой слой данных, отвечающий за хранение и предоставление информации. В Django каждая модель соответствует одной таблице в базе данных и наследуется от класса django.db.models.Model.

2. Поля моделей и их типы

Каждое поле в модели - это экземпляр соответствующего класса Field, который определяет тип данных столбца в БД:

- CharField - строка фиксированной длины (обязателен параметр max_length).

- TextField - текстовое поле большого объема.

- DateTimeField - дата и время.

- ImageField - поле для загрузки изображений (требует библиотеку Pillow).

- BooleanField - логическое поле (True/False).

Параметры полей

- blank=True - поле может быть пустым в формах.

- null=True - поле может хранить NULL в базе данных.

- auto_now_add - автоматически устанавливает текущую дату/время при создании объекта.

- auto_now - автоматически обновляет дату/время при сохранении объекта.

- upload_to - указывает подкаталог для загрузки файлов (медиафайлов).

3. Миграции

Миграции (migrations) - это способ Django отслеживать и применять изменения в структуре таблиц. Команды для работы с миграциями:

- python manage.py makemigrations - создать файлы миграций на основе изменений в моделях.

- python manage.py migrate - применить миграции к базе данных.

- python manage.py sqlmigrate <app> <migration_name> - показать SQL-запрос, который будет выполнен.

4. ORM Django (Object-Relational Mapping)

ORM позволяет взаимодействовать с базой данных через Python-код, без написания SQL. Основные методы работы с записями (CRUD):

- Create:
  - Через конструктор: News(title='...') + save().
  - Через менеджер объектов: News.objects.create(title='...').
- Read:
  - all() - получить все записи.
  - filter(kwargs) - отфильтровать записи.
  - get(kwargs) - получить одну запись (если запись не найдена или найдено несколько - возникнет исключение).
  - exclude(kwargs) - исключить записи по условию.
- Update:
  - Изменить атрибуты объекта и вызвать save().
- Delete:
  - Вызвать метод delete() у объекта.

5. Медиафайлы

Для корректной работы с загружаемыми файлами (изображениями) в Django необходимо определить в settings.py:

- MEDIA_ROOT - абсолютный путь к каталогу, где будут храниться файлы.

- MEDIA_URL - URL-префикс для доступа к медиафайлам.

В режиме отладки (DEBUG = True) нужно добавить маршрут в корневой urls.py для обслуживания медиафайлов через django.conf.urls.static.static().

Методика выполнения

Модели хранят данные не у себя в файлах, а хранят в базах данных (БД). Django работает sqllite3. И для управления понадобится любой менеджер БД. Воспользуемся менеджером sqlitestudio_x64-3.4.3, портативной версией, которую вы можете скачать и без установки использовать в проекте.

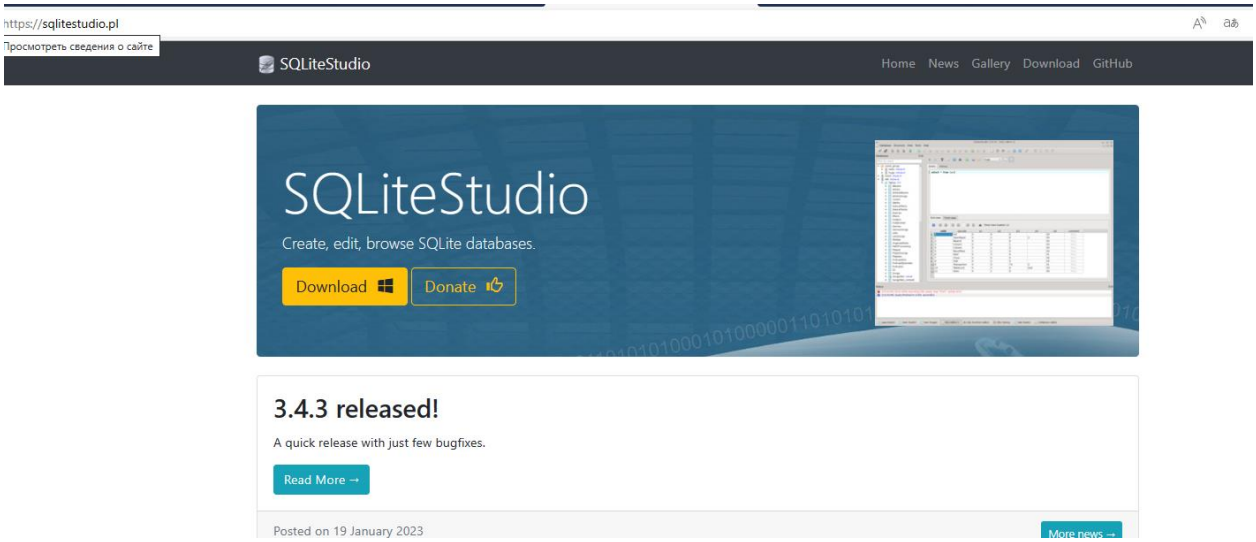


Рисунок 1

Выбрать версию менеджера и установить на своем компьютере с помощью файла SQLiteStuduo.exe (рисунок 3).

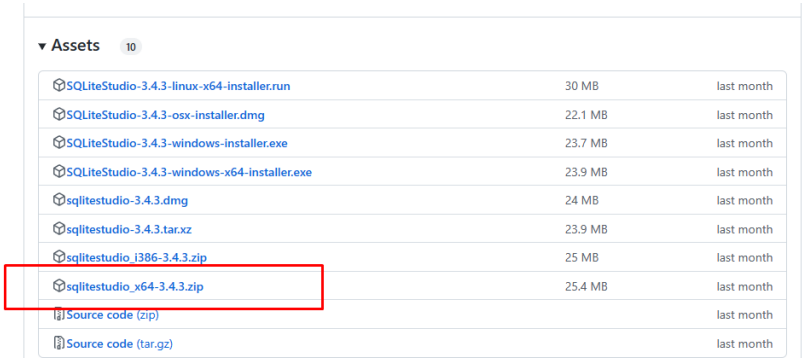


Рисунок 2

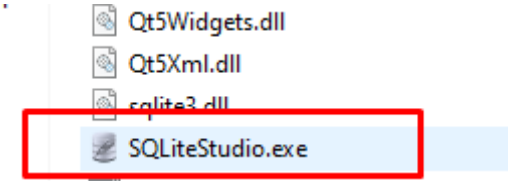


Рисунок 3

Добавляем БД (рисунок 4). В поле файл необходимо указать путь файла БД нашего проекта db.sqlite3.

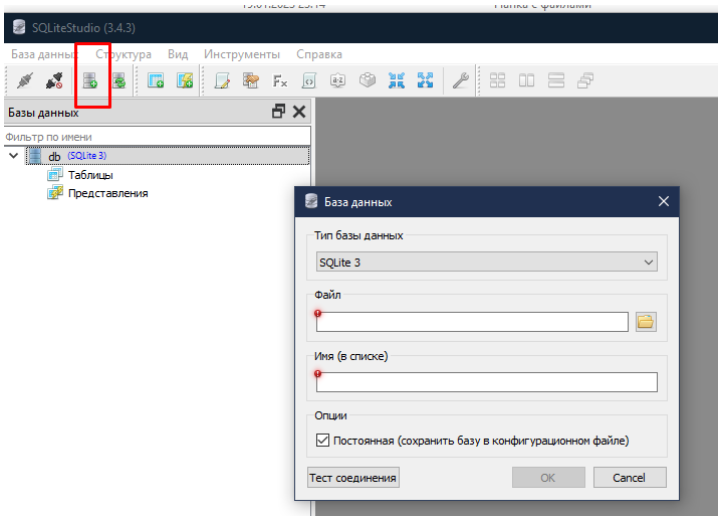


Рисунок 4

Подключаем БД (рисунок 5)

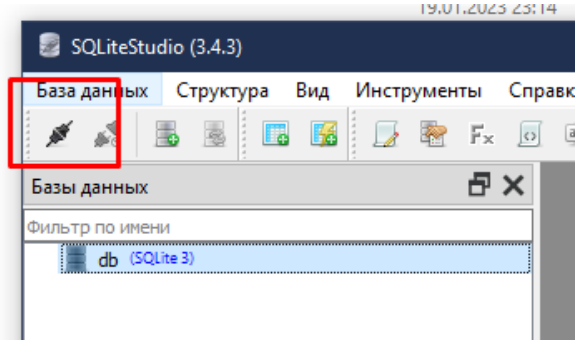


Рисунок 5

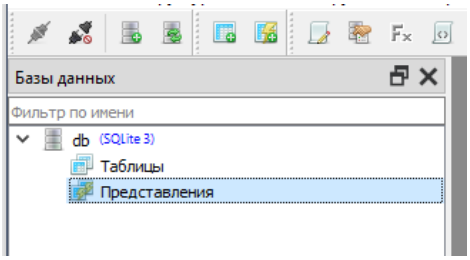


Рисунок 6

Пока база данных у нас пустая. Модели в Django описываются классом внутри соответствующего файла models в приложении news.

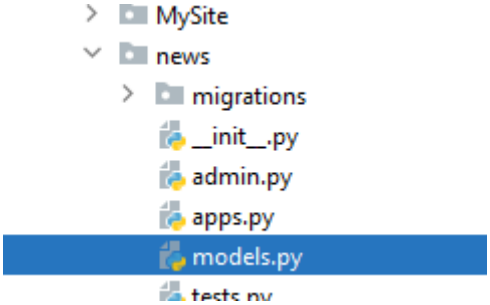


Рисунок 7

Здесь будем писать свой класс, который является подклассом django.db import models и получать все методы для работы с данными.

В этом классе мы будем описывать атрибуты (свойства) класса в модели, которые являются обязательным. Атрибуты – это соответствующие поля таблицы БД. В нашем случае это таблица news.

Давайте определим поля нашей таблицы.

Название поля	**Атрибут**	**Назначение поля**
title	Текстовое	Название статьи
content	Текстовое	Содержание статьи

create_at	Дата/Время	Дата и время создания статьи
updated	Дата/Время	Дата и время обновления статьи
photo	Тип картинка	
is_published	логическое	Опубликована статья/ не опубликована статья

При выборе названий полей, то нельзя использовать зарезервированные слова.

Каждый атрибут должен быть экземпляром соответствующего класса Field

Класс создается с помощью class имя(models.Model): (описание полей с типами)

Создаем модель

```
class News(models.Model):
    title=models.CharField(max_length=150)
    content=models.TextField(blank=True)
    created_at=models.DateTimeField(auto_now_add=True)
    updated_at=models.DateTimeField(auto_now=True)
    photo=models.ImageField(upload_to='photos/%Y/%m/%d')
    is_published=models.BooleanField(default=True)
```

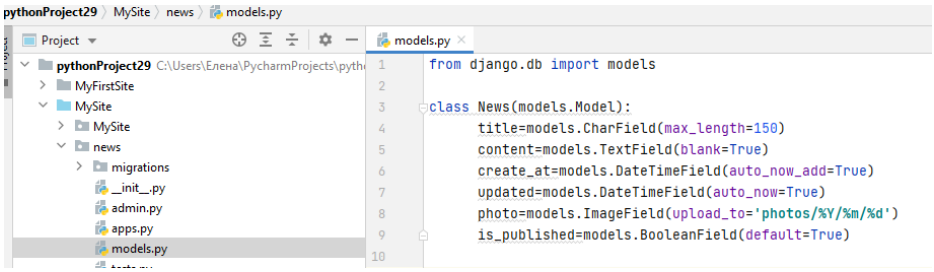


Рисунок 8

ID мы не будем описывать. Он создается автоматически Django.

Опишем параметры соответствующих полей:

auto_now_add - автоматически установите для поля значение «Сейчас» при первом создании объекта.

auto_now - автоматически устанавливайте для поля значение «Сейчас» при каждом сохранении объекта

upload_to - является форматирование strftime(), четырехзначный год, двузначный месяц и равен двузначный день. Если вы загрузите файл 15 января 2023 г., он будет сохранен в каталог .'/home/media"photos/%Y/%m/%d"%Y/%m/%d"%Y"%m"%d'/home/media/photos/2023/01/15

Blank=True - означает, что это поле обязательное для заполнения.

После создания модели необходимо перейти к созданию файла миграции (рисунок 9).

Миграции - это способ, которым Django распространяет изменения, которые вы вносите в модели (добавление поля, удаление модели и т. Д.) В схеме базы данных. Они разработаны так, чтобы быть почти автоматическими, но вам нужно знать, когда создавать миграции, когда их запускать, а также общие проблемы, с которыми вы можете столкнуться.

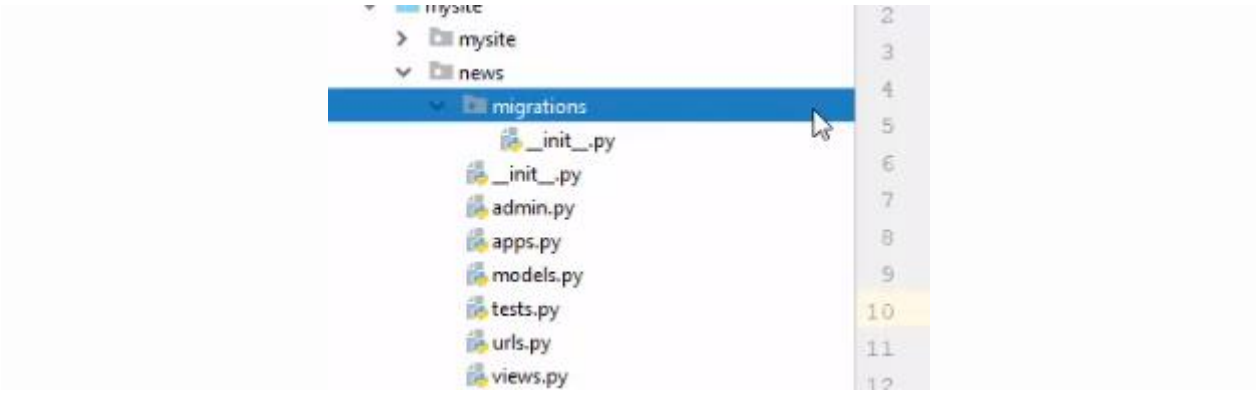


Рисунок 9

Создаются с помощью команды makemigrations, которую выполняем в терминале. Но прежде чем проводить миграцию, необходимо убедиться, что все поля заданы корректно и подключены все библиотеки. Например, если будет обнаружена ошибка такого рода как на рисунке 10, то прежде перед

создание миграции необходимо установить библиотеку Pillow для работы с изображениями. И пока не будут установлены нужные библиотеки миграцию провести не возможно.

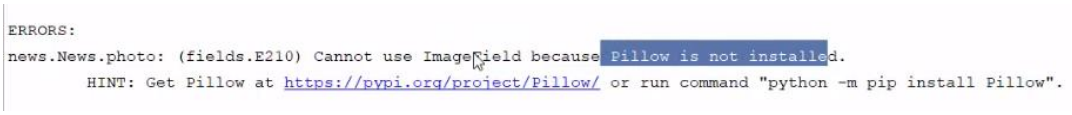


Рисунок 10

Для установки этой библиотеки воспользуемся командой pip install Pillow (рисунок 11)

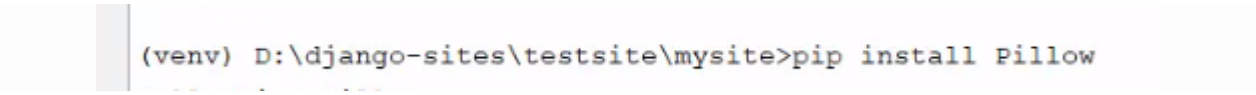


Рисунок 12

Перезапустите сервер.

В терминале введем команду создания миграции makemigrations, которая отслеживает все изменения в моделях и создает миграции для всех этих изменений. Строка команды следующая (рисунок 13)

python manage.py makemigrations

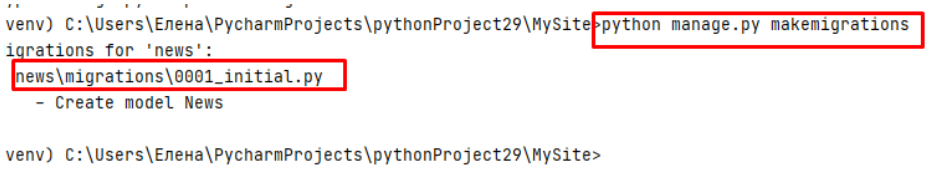


Рисунок 13

В результате буден создан файл 0001_initial.py (рисунок 14)

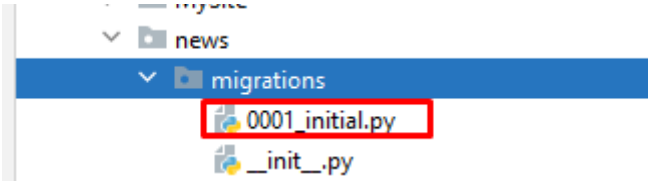


Рисунок 14

А также создана модель News (рисунок 15)

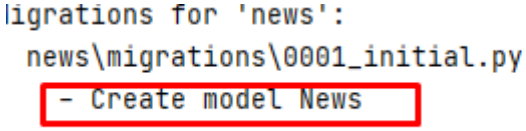


Рисунок 15

В файле 0001_initial.py мы можем наблюдать эту миграцию (рисунок 16).



Рисунок 16

Как вы видите, id, который мы не указывали в модели был создан автоматически.

Посмотрим SQL запрос, который может быть выполнен при создании таблица в БД.

В терминале выполним следующий запрос (рисунок 17)

python manage.py sqlmigrate news 0001



Рисунок 17

В пакете конфигурации в файле settings можно увидеть какая БД в настоящий момент подключена (рисунок 18) в виде словаря DATABASES .



Рисунок 18

Как мы видим, в настоящем проекте подключена БД sqlite3.

В словаре используется константа BASE_DIR, которая прописывается в этом файле и вычисляется местонахождение вашей БД, указывает на корень нашего проекта (рисунок 19).

```
from pathlib import Path

# Build paths inside the project like this: BASE_DIR / 'subd
BASE_DIR = Path(__file__).resolve().parent.parent
```

Рисунок 19

Теперь перейдем к выполнению миграций. При старте сервера в терминале можно увидеть сообщение (рисунок 20), в котором говориться, что у нас не применено 18 миграций. Это миграции, которые есть в проекте Django за исключением нашей и которые отвечают за админку, сессии и прочее.

```
You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, contenttypes, sessions.
```

Рисунок 20

Для того, чтобы выполнить миграции используется команда

python manage.py migrate

Выполняем ее (рисунок 21)

```
(venv) C:\Users\Елена\PycharmProjects\pythonProject29\MySite>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, news, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying news.0001_initial... OK
  Applying sessions.0001_initial... OK
```

Рисунок 21

ОК в конце каждой строки указывает, что все миграции выполнены успешно в том числе и миграция нашей модели, которая выполнилась последней.

Обновим структуру нашей БД и увидим, что здесь появились таблицы (рисунок 22) среди которых мы видим таблицу news_news.

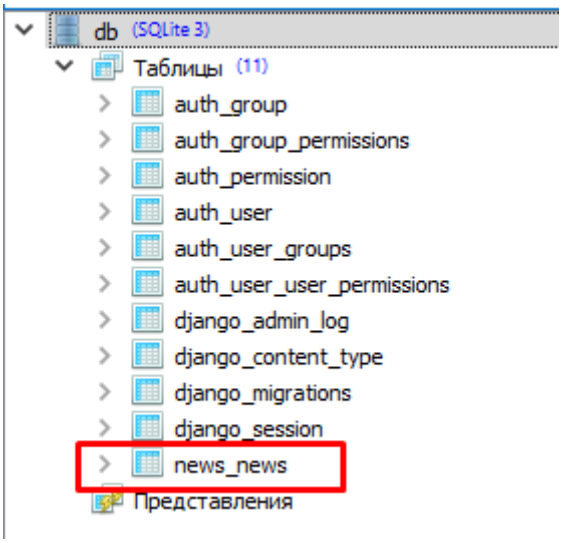


Рисунок 22

Выбрав в дереве таблиц таблицу news_news, можно просмотреть поля таблицы, соответствующие полям нашей модели (рисунок 23).

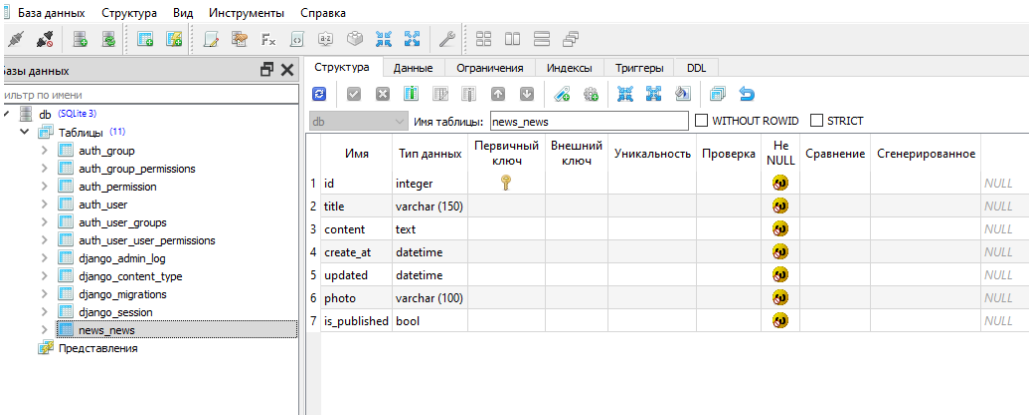


Рисунок 23

А также SQL запрос, который был использован для этой таблицы (рисунок 24)

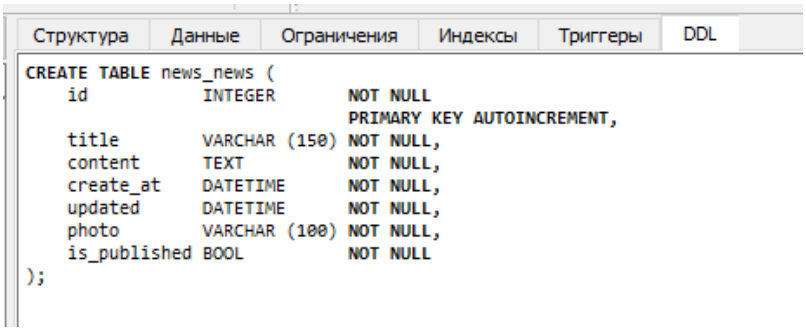


Рисунок 24

Необходимо обратить внимание на поле photo, которое имеет тип varchar и изображения, которого нам необходимо куда-то сохранять.

Для настройки этого поля необходимо настроить константы MEDIA_ROOT и MEDIA_URL. Эти файлы нужны для того, чтобы указать, куда загружать файлы.

Переходим в файл настроек settings и в конце файла дописываем константы (рисунок 25)

```
MEDIA_ROOT=os.path.join(BASE_DIR, 'media')
MEDIA_URL= '/media/'
```

```
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

MEDIA_ROOT=os.path.join(BASE_DIR, 'media')
MEDIA_URL= '/media/'
```

Рисунок 25

Для того, чтобы файлы-картинки загружались на странице сайте (отображались), то необходимо создать соответствующий маршрут, который будет указывать на загруженные файлы. Для этого используется функция static из модуля Django. Эта функция создаст маршрут, по которому маршрутизатор передаст при запросе файлов управление специальному контроллеру Django, который отдаст файл-фото для просмотра. Но этот маршрут необходим только в отладочном режиме, то есть только при разработке сайта. При переводе на реальный сервер этот маршрут нам уже будет не нужен и файлы будут обслуживаться самим веб-сервером.

В файле settings есть логическая константа DEBUG = True, которой присвоено значение True. Это означает, что ведется разработка. После окончания разработки True необходимо заменить на False.

Переходим в корневой файл urls.

Исходя из константы DEBUG мы обращаемся к модулю settings для того, чтобы указать, путь для загрузки медиа в отладочном режиме через условие. При этом потребуется подключение библиотеки, которую можно импортировать вручную или нажать Alt+Enter и выбрать импортируемую библиотеку (рисунок 25).

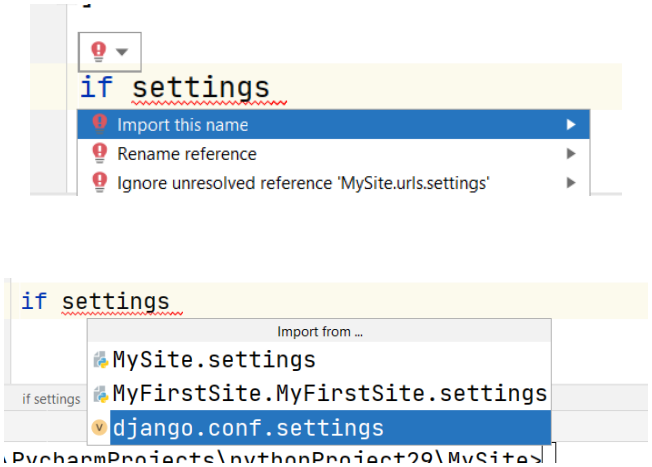


Рисунок 25

И дописываем собственный маршрут, который сформирует функция static, которую тоже нужно импортировать (рисунок 26).

uments

```
from django.conf import settings
from django.contrib import admin
from django.urls import path, include
from django.conf.urls.static import static
from news.views import *


urlpatterns = [
    path('admin/', admin.site.urls),
    path('news/',include('news.urls')),

]

if settings.DEBUG:
    urlpatterns+=static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Рисунок 26

По этому маршруту будем получать медиа-файлы.

Для заполнения базы данных, которая была создана в предыдущей работе (рисунок 27), необходимо добавить данные через админку.

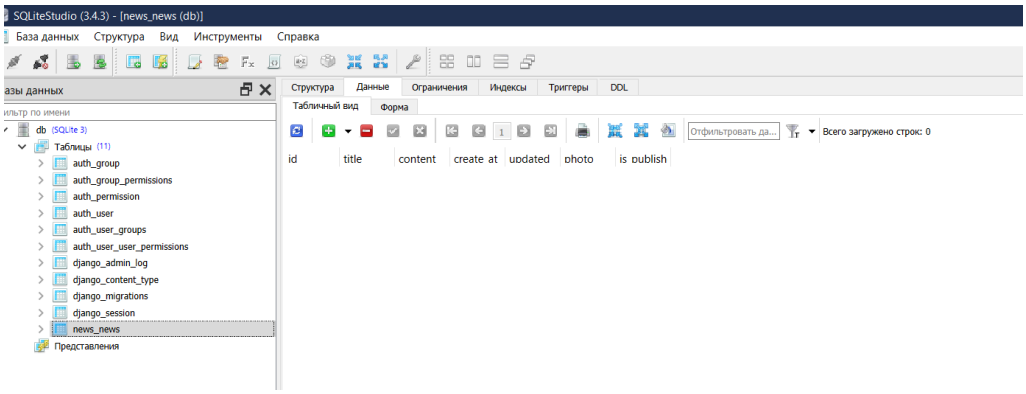


Рисунок 27

И для этого необходимо создать суперпользователя для админки, который обладает всеми правами и может работать в админке. Но пока научимся добавлять данные из консоли Django для ознакомления работы с API для работы с моделями.

У Django есть свое API для работы с моделями, есть ORM.

Чтобы протестировать модель, воспользуемся консолью Django. Консоль Django – это таже самая интерактивная консоль Python, где можно писать код. Откроем ее. Для этого необходимо перейти в папку нашего проекта в Pycharm и выполнить команду shell через обращение к manage.py (рисунок 28)

python manage.py shell

```
(venv) C:\Users\Елена\PycharmProjects\pythonProject29\MySite>python manage.py shell
Python 3.11.2 (tags/v3.11.2:878ead1, Feb  7 2023, 16:38:35) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>>
```

Рисунок 28

Как видим, после ввода становится активна интерактивная консоль InteractiveConsole.

Для того чтобы работать с моделью нам нужно ее импортировать из приложения, где она находится. Наша модель находиться в приложении news.

Обращайте внимание на регистры. Модели принято называть в верхнем регистре (рисунок 29).

```
>>> from news.models import News
>>>
```

Рисунок 29

Запишем данные в таблицу. Запись модели создаются вызовом ее конструктора news и передаче в этот самый конструктор именованных параметров с соответствующими атрибутами модели. Просмотрим, какие параметры нужно передавать. Предполагается поле title, которое должно быть обязательным, поле Content с атрибутом blank=True, который указывает, что данное поле является не обязательным для заполнения. Поле created_add мы не трогаем. При единоразовой записи, это поле будет заполнено текущим временем. Поле updated_at будет заполнено при создании записи и при каждом ее обновлении. Здесь также будет заполнено текущим временем. Поле photo используется для загрузки картинки, конечно мы ее загружать из консоли не будем и пока тоже не будем трогать это поле и не смотря, что здесь не поставлен атрибут blank=True, это поле является обязательным для заполнения, но если мы работаем напрямую через консоль, то можем тоже пока не заполнять и Django сам будет формировать SQL-запрос и появление пустой строки, тоже будет являться значением. Поле is_published является логическим полем и при создании будет проставляться 1. Итак, по факту будут заполнены только два поля title, Content.

Вызовем конструктор нашей модели

News(title='Новость 1', content='Контент новости 1')

```
>>> from news.models import News
>>> News(title='Новость 1', content='Контент новости 1')
<News: News object (None)>
>>>
```

Рисунок 30

В результате получим <News: News object (None)> (рисунок 31)

Давайте посмотрим таблицу в БД и после обновления кнопкой, то увидим, что данные не добавились. Это происходит так, потому что эти данные нужно сохранить. На данный момент мы просто заполнили конструктор данными, но не выгрузили в таблицу. За сохранение данных отвечает специальный метод модели, это метод save. И здесь стоит обратить внимание на то, что модели по умолчанию в Django являются «ленивыми». Это означает, что мы можем формировать sql запрос с помощью API модели, можем делать в нескольких строках кода, вызывая методы и с каждой строкой ни чего происходит не будет до тех пор пока не запустим запрос на исполнение. И в случае записи данных делается это с помощью метода save, в случае чтения данных это делается в момент запроса этих самых данных. Т.е. сами запросы сами по себе не выполняются и выполняются только тогда, когда нужен результат этого запроса.

В нашем случае нужно сохранить эти данные, поэтому давайте создадим некоторую переменную news1 (news1=_) и присвоим ей результат предыдущего выражения в консоли с помощью псевдопеременной с нижним подчеркиванием, в которой хранится предыдущий результат.

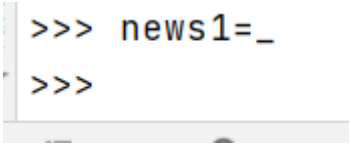


Рисунок 32

И если выведем эту переменную, то в ней буде объект ранее полученный

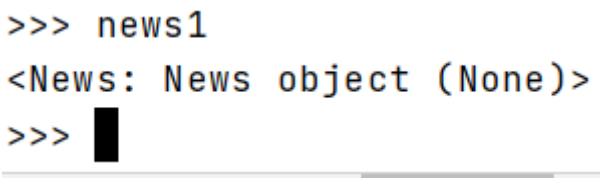


Рисунок 33

Non – это номер объекта, т.е. id, который пока что у нас его нет и не будет до тех пор пока запись не будет сохранена. Id назначается автоматически.

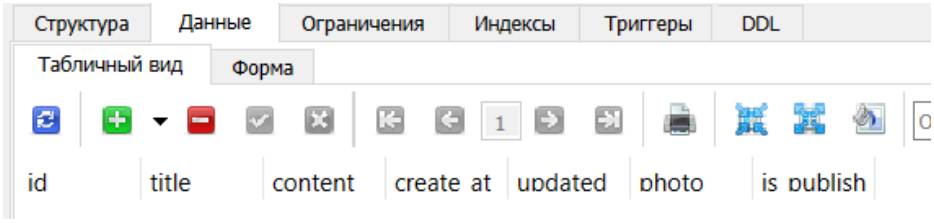


Рисунок 34

Но все свойства, например, news1.title у нас есть (рисунок 34)

```
>>> news1.title
'Новость 1'
>>>
```

Рисунок 35

Или news1.content (рисунок 36)

```
>>> news1.content
'Контент новости 1'
```

Рисунок 36

Если обратимся к полю id, то увидим пустую строку, после ввода команды (рисунок 37)

```
>>> news1.id
>>>
```

Рисунок 37

Сохраним эту запись с помощью метода save(): news1.save().

```
>>> news1.save()
>>> news1.id
1
```

Рисунок 38

После ввода этой команды сохранения и вывода, например, id, мы увидим, что поле id уже имеет свое уникальное значение «1». Это означает, что Django сохранил эту запись.

Если мы обновим запись в SQLiteStudio, то увидим уже эту запись в БД (рисунок 39)

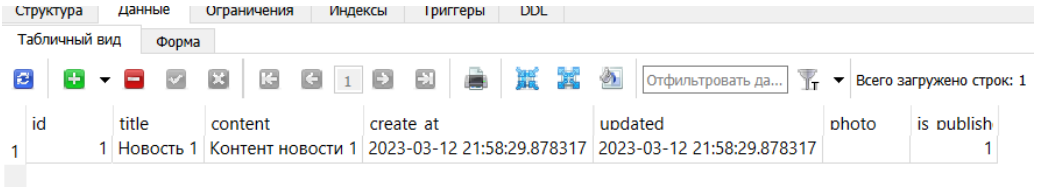


Рисунок 39

Поле title и content заполнились нашими данными, поле create_at и update_at заполнились текущими данными, поле is_publish – заполнилось логической 1, признаком публикации, поле photo – пустое.

Можно просмотреть выполненные запросы. Для этого на нужно из модуля Django.db импортировать connection и просмотреть его свойства queries

from django.db import connection

connection.queries

```
>>> from django.db import connection
>>> connection.queries
[{'sql': 'INSERT INTO "news_news" ("title", "content", "create_at", "updated", "photo", "is_published") VALUES (\'Новость 1\', \'Контент новости 1\', \'2023-03-12 21:
58:29.878317\', \'2023-03-12 21:58:29.878317\', \'\', 1) RETURNING "news_news"."id"', 'time': '0.000'}]
>>>
```

Рисунок 40

В результате мы получаем список словарей с двумя ключами ‘sql’ – это запрос и ‘time’ – это время, которое потребовалось на его выполнение.

У news появилось такое свойство как pk, которое равно свойству id.

```
>>> news1.pk
1
>>>
```

Рисунок 41

В Django многое работает на уровне соглашения, т.е. так называемые рекомендации как именовать то или иное выражение и в том числе, есть такое свойство pk (сокращение от слов primary key) и это то, как мы можем обращаться к полю id через универсальное свойство pk и хранить в себе ключевое значение записи.

Создадим и сохраним еще одну запись:

news2=News(title='Новость 2', content='Контент новости 2')

```
>>> news2=News(title='Новость 2', content='Контент новости 2')
>>> news2.save()
>>>
```

Рисунок 42

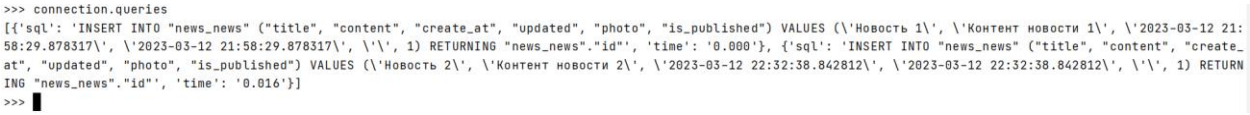


Рисунок 43

Для очистки консоли можно воспользоваться командой Ctrl+L.

Рассмотрим еще один вариант добавления записи. Можно вызвать модель без параметров и дальше заполнить ее параметрами. (рисунок 17)

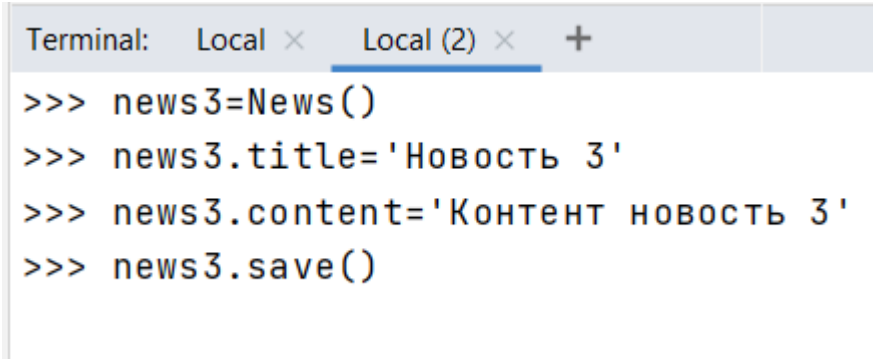
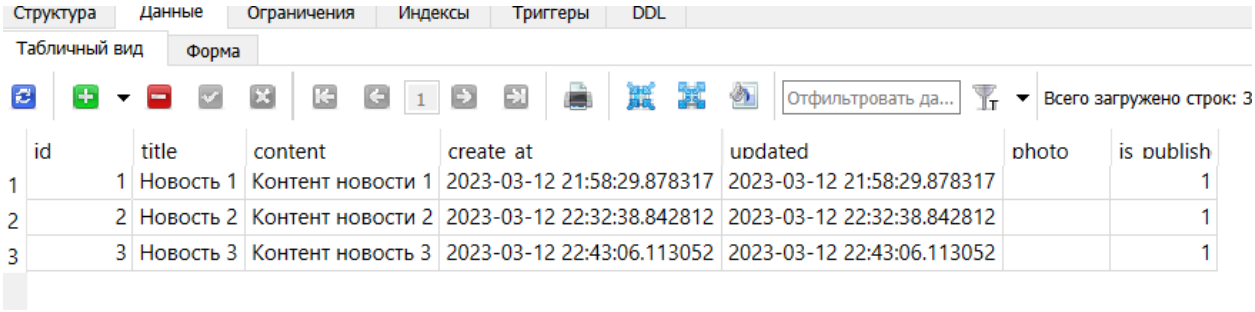


Рисунок 44



	id	title	content	create at	updated	photo	is publish
1	1	Новость 1	Контент новости 1	2023-03-12 21:58:29.878317	2023-03-12 21:58:29.878317		1
2	2	Новость 2	Контент новости 2	2023-03-12 22:32:38.842812	2023-03-12 22:32:38.842812		1
3	3	Новость 3	Контент новость 3	2023-03-12 22:43:06.113052	2023-03-12 22:43:06.113052		1

Рисунок 45

После обновления мы увидим в таблице уже три записи.

Рассмотри третий вариант добавления записи. Для этого воспользуемся атрибутом класса Objects. Данный атрибут хранит диспетчер записей и позволяет манипулировать записями модели. Через этот диспетчер мы можем не только сохранять записи, но и получать их от туда. Данный диспетчер является экземпляром специального класса manader.

news4=News.objects.create(title='Новость 4', content='Контент новости 4')

После нажатия Enter, запись будет автоматически сохранена и не нужно будет вызывать метод save(). Убедимся, если обратимся к свойству pk

```
>>> news4=News.objects.create(title='Новость 4', content='Контент новости 4')
>>> news4.pk
4
```

Табличный вид | Форма

Отфильтровать да... | Всего загружено строк: 4

	id	title	content	create at	updated	photo	is publish
1	1	Новость 1	Контент новости 1	2023-03-12 21:58:29.878317	2023-03-12 21:58:29.878317		1
2	2	Новость 2	Контент новости 2	2023-03-12 22:32:38.842812	2023-03-12 22:32:38.842812		1
3	3	Новость 3	Контент новость 3	2023-03-12 22:43:06.113052	2023-03-12 22:43:06.113052		1
4	4	Новость 4	Контент новости 4	2023-03-12 22:50:11.880458	2023-03-12 22:50:11.880458		1

Рисунок 46

Метод create() создает запись и сразу ее возвращает.

Давайте создадим еще одну запись и посмотрим как возвращает функция create(). Создадим новую запись следующего вида

News.objects.create(title='Новость 5', content='Контент новости 5')

И после нажатия Enter, мы видим <News: News object (5)>

```
>>> News.objects.create(title='Новость 5', content='Контент новости 5')
<News: News object (5)>
>>>
```

Рисунок 47

Как на рисунке 20 мы видим, что нам вернулось не None, а значение 5.

Таким образом, мы рассмотрели операцию create технологии CRUD. Перейдем к рассмотрению операции read, т.е. получение данных как одна из наиболее частых операций.

Для получения всех записей используется метод all(). Это диспетчер записи атрибута Objects, который вернет набор записей, являющихся экземпляром класса QuerySet.

При этом не забудьте активировать интерактивную консоль InteractiveConsole.

Попробуем получить все записи из модели News.

News.objects.all()

Модель вернет все наши записи (рисунок 48)

```
>>> News.objects.all()
<QuerySet [<News: News object (1)>, <News: News object (2)>, <News: News object (3)>, <News: News object (4)>, <News: News object (5)>]>
>>>
```

Рисунок 48

В таком формате по Django мы видим только название объекта и его id. Необходимо посмотреть с значения и здесь будем использовать метод, который возвращает строковое представление объекта. Т.е. выведем записи, в нашем случае, с названиями новостей.

В файле molels.py опишем этом метод с помощью функции (рисунок 49)

```
def __str__(self):
    return self.title
```

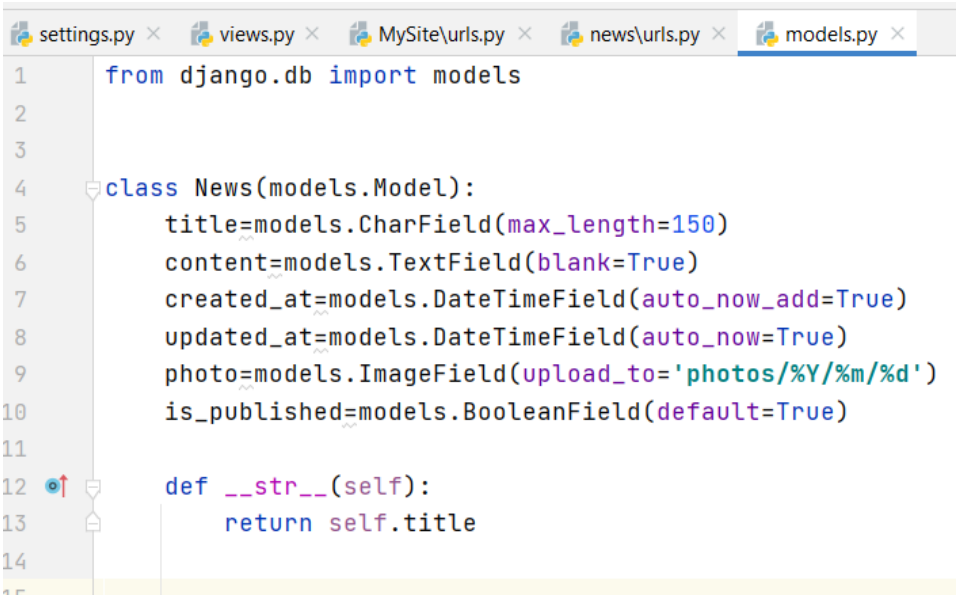


Рисунок 49

И теперь при распечатке данного объекта будет работать метод __str__(), которое возвращает строковое представление объекта, в нашем примере, это название новости.

И попробуем еще раз выполнить команду News.objects.all(), но при этом изменения вступят в силу нужно выйти из консоли Django с помощью команды exit() (рисунок 50)

```
<QuerySet [<News: News object (1)>, <News: News object (2)>, <News: News objec
>>> exit()

(venv) C:\Users\Елена\PycharmProjects\pythonProject29\MySite>
```

Рисунок 50

И еще раз зайти python manage.py shell

После перезагрузки снова импортируем модель

from news.models import News

и записываем News.objects.all() (рисунок 51)

```
>>> News.objects.all()
<QuerySet [<News: Новость 1>, <News: Новость 2>, <News: Новось 3>, <News: Новость 4>, <News: Новость 5>]>
>>>
```

Рисунок 51

Как видим, вывелись записи с названиями новостей.

Поскольку был получен набор значений то его можно обработать в цикле. Сохраним этот набор в переменную news=_

```
>>> news=_
>>> news
<QuerySet [<News: Новость 1>, <News: Новость 2>, <News: Новось 3>, <News: Новость 4>, <News: Новость 5>]>
>>>
```

Рисунок 52

Например, нам необходимо вывести список новостей с признаком опубликована новость или нет (рисунок 53).

for item in news:

    print(item.title, item.is_published)

```
>>> for item in news:
...     print(item.title, item.is_published)
...
Новость 1 True
Новость 2 True
Новось 3 True
Новость 4 True
Новость 5 True
>>>
```

Рисунок 54

Но чаще всего весь список может быть и не нужен, а нужны по какому-нибудь критерию записи. Для этого используется метод filter().

Предварительно добавим пару записей с одинаковыми заголовками, но с разным контентом (рисунок 55).

```
>>> News.objects.create(title='Новость 6', content='Контент новости 6')
<News: Новость 6>
>>> News.objects.create(title='Новость 6', content='Контент новости 7')
<News: Новость 6>
>>>
```

Рисунок 55

И обновим БД (рисунок 56)

Структура | Данные | Ограничения | Индексы | Триггеры | DDL

Табличный вид | Форма

Отфильтровать да... | Всего загружено строк: 7

	id	title	content	created_at	updated_at	photo	is_publishec
1	1	Новость 1	Контент новости 1	2023-03-14 09:49:29.507892	2023-03-14 09:49:29.507892		1
2	2	Новость 2	Контент новости 2	2023-03-14 09:53:47.536519	2023-03-14 09:53:47.536519		1
3	3	Новось 3	Контент новости 3	2023-03-14 09:58:34.912208	2023-03-14 09:58:34.912208		1
4	4	Новость 4	Контент новости 4	2023-03-14 09:59:06.663398	2023-03-14 09:59:06.663398		1
5	5	Новость 5	Контент новости 5	2023-03-14 10:00:07.824249	2023-03-14 10:00:07.824249		1
6	6	Новость 6	Контент новости 6	2023-03-14 10:43:03.968536	2023-03-14 10:43:03.968536		1
7	7	Новость 6	Контент новости 7	2023-03-14 10:43:20.793224	2023-03-14 10:43:20.793224		1

Рисунок 56

И теперь вернем список записей с названием Новость 6 с использованием метода filter() (рисунок 57).

```
>>> News.objects.filter(title='Новость 6')
<QuerySet [<News: Новость 6>, <News: Новость 6>]>
>>>
```

Рисунок 57

И теперь, как видим, QuerySet состоит только из двух нужных нам записей.

В других случаях, может возникнуть необходимость вывода только одной записи, которую как правило выбирают по id с помощью метода get().

News.objects.get(pk=5)

```
>>> News.objects.get(pk=5)
<News: Новость 5>
>>>
```

Рисунок 58

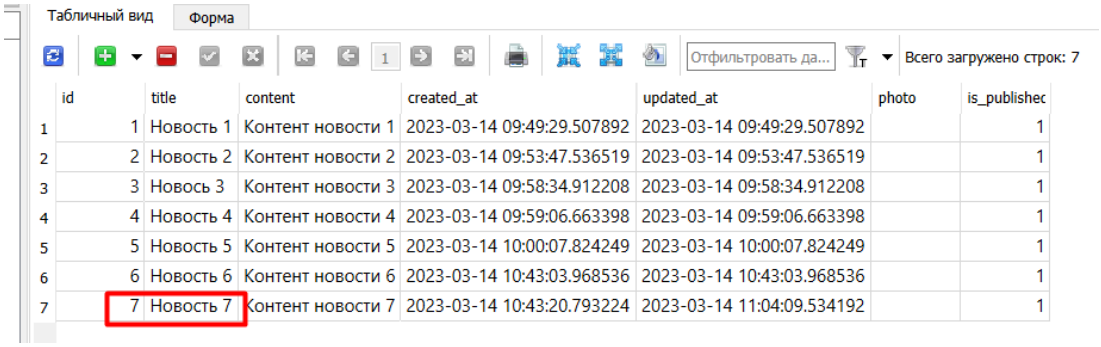
Следующий процесс – это обновление записи.

Чтобы обновить запись необходимо ее получить. Давайте обновим запись с id=7 и переименуем название новости (рисунок 59).

```
>>> news7=_
>>> news7
<News: Новость 7>
>>> news7.title='Новость 7'
>>> news7.save()
>>>
```

Рисунок 59

Обновляем БД (рисунок 60).



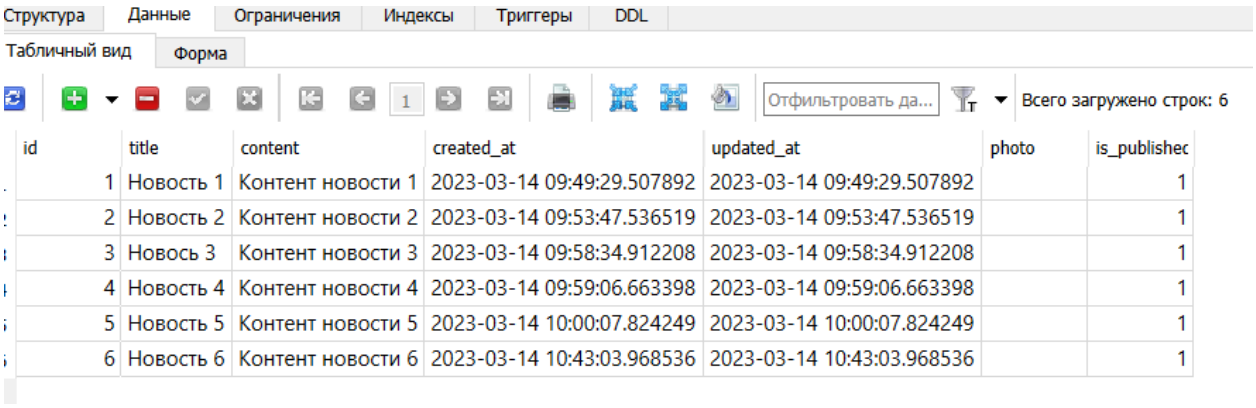
	id	title	content	created_at	updated_at	photo	is_publishec
1	1	Новость 1	Контент новости 1	2023-03-14 09:49:29.507892	2023-03-14 09:49:29.507892		1
2	2	Новость 2	Контент новости 2	2023-03-14 09:53:47.536519	2023-03-14 09:53:47.536519		1
3	3	Новось 3	Контент новости 3	2023-03-14 09:58:34.912208	2023-03-14 09:58:34.912208		1
4	4	Новость 4	Контент новости 4	2023-03-14 09:59:06.663398	2023-03-14 09:59:06.663398		1
5	5	Новость 5	Контент новости 5	2023-03-14 10:00:07.824249	2023-03-14 10:00:07.824249		1
6	6	Новость 6	Контент новости 6	2023-03-14 10:43:03.968536	2023-03-14 10:43:03.968536		1
7	7	Новость 7	Контент новости 7	2023-03-14 10:43:20.793224	2023-03-14 11:04:09.534192		1

Рисунок 60

Рассмотрим теперь процесс удаления delete(). Для примера удалим запись с id 7 (рисунок 61).

```
>>> news7=News.objects.get(pk=7)
>>> news7.delete()
(1, {'news.News': 1})
>>>
```

Рисунок 61


Структура | Данные | Ограничения | Индексы | Триггеры | DDL

Табличный вид | Форма

Отфильтровать да... Всего загружено строк: 6

id	title	content	created_at	updated_at	photo	is_publishec
1	Новость 1	Контент новости 1	2023-03-14 09:49:29.507892	2023-03-14 09:49:29.507892		1
2	Новость 2	Контент новости 2	2023-03-14 09:53:47.536519	2023-03-14 09:53:47.536519		1
3	Новось 3	Контент новости 3	2023-03-14 09:58:34.912208	2023-03-14 09:58:34.912208		1
4	Новость 4	Контент новости 4	2023-03-14 09:59:06.663398	2023-03-14 09:59:06.663398		1
5	Новость 5	Контент новости 5	2023-03-14 10:00:07.824249	2023-03-14 10:00:07.824249		1
6	Новость 6	Контент новости 6	2023-03-14 10:43:03.968536	2023-03-14 10:43:03.968536		1

Рисунок 62

После обновления БД видим, что этой записи уже нет.

Рассмотрим еще метод exclude() – исключение не нужных записей.

News.objects.exclude(title='Новость 4')

```
>>> News.objects.exclude(title='Новость 4')
<QuerySet [<News: Новость 1>, <News: Новость 2>, <News: Новось 3>, <News: Новость 5>, <News: Новость 6>]>
>>>
```

Рисунок 63

Как видим из рисунка 35, запись 'Новость 4' была исключена

Варианты индивидуальных заданий

Каждый вариант предполагает создание модели (или доработку существующей), выполнение миграций, наполнение базы данных через консоль Django (shell) и проверку работы CRUD-операций.

Общее задание:

Создать проект и приложение в соответствии с темой варианта. Для общего задания использовать лабораторную работу 1.

Вариант 1. Блог

- Модель: Post (поля: title, content, author, created_at, is_published).
- Добавить поле views_count (целое число, по умолчанию 0).
- Написать метод __str__, возвращающий заголовок поста.
- Вывести все посты, у которых is_published=True.

Вариант 2. Каталог товаров

- Модель: Product (поля: name, price, description, in_stock (bool), created_at).

- Добавить поле quantity (целое число, по умолчанию 1).

- Вывести список товаров дешевле 1000 руб.

Вариант 3. Комментарии

- Модель: Comment (поля: user_name, email, text, created_at, is_moderated).

- Добавить поле rating (целое число от 1 до 5).

- Вывести все комментарии, прошедшие модерацию.

Вариант 4. Задачи (To-Do)

- Модель: Task (поля: title, description, deadline, is_completed, priority (целое число)).

- Добавить поле category (строка, максимум 50 символов).

- Вывести все невыполненные задачи, отсортированные по приоритету.

Вариант 5. События (Event)

- Модель: Event (поля: name, date, location, max_participants, is_active).

- Добавить поле organizer (строка, максимум 100 символов).

- Вывести все активные события на текущую дату.

Вариант 6. Отзывы (Review)

- Модель: Review (поля: product_name, author, rating (целое), text, created_at).

- Добавить поле useful_count (целое число, по умолчанию 0).

- Найти все отзывы с рейтингом выше 3.

Вариант 7. Сотрудники (Employee)

- Модель: Employee (поля: full_name, position, hire_date, salary, is_active).

- Добавить поле phone (строка).

- Вывести всех активных сотрудников, отсортированных по дате приёма.

Вариант 8. Книги (Book)

- Модель: Book (поля: title, author, year, isbn, is_available).

- Добавить поле pages (целое число).

- Найти все книги, изданные после 2000 года.

Вариант 9. Курсы валют

- Модель: CurrencyRate (поля: currency_code, rate, date).

- Добавить поле change_percent (число с плавающей точкой).

- Вывести все курсы на определённую дату.

Вариант 10. Заказы (Order)

- Модель: Order (поля: customer_name, order_date, total_amount, is_paid).

- Добавить поле delivery_address (текстовое поле).

- Найти все оплаченные заказы.

Контрольные вопросы

1. Что такое модель в Django и для чего она используется?

2. Какие типы полей вы знаете? Приведите примеры.

3. Чем отличаются параметры blank=True и null=True?

4. Что такое миграции и зачем они нужны? Опишите основные команды для работы с ними.

5. Как создать суперпользователя для доступа к административной панели Django?

6. Что такое ORM? Перечислите основные методы для выполнения операций CRUD.

7. В чем разница между методами filter() и get()?

8. Как настроить загрузку изображений (медиафайлов) в проекте Django?

9. Для чего нужен метод __str__ в модели?

10. Что произойдёт, если выполнить python manage.py makemigrations, но не выполнить migrate?

Критерии оценивания

Критерий	**Описание**	**Макс. балл**
Правильность создания модели	Модель создана в соответствии с заданием, корректно выбраны типы полей, указаны необходимые параметры	2
Выполнение миграций	Созданы и применены миграции, таблица появилась в БД	1
Настройка медиафайлов (если требуется)	Правильно прописаны MEDIA_ROOT и MEDIA_URL, настроен маршрут в urls.py	1
Работа с ORM (консоль Django)	Успешное создание, чтение, обновление и удаление записей через shell.	3

Использование фильтрации и исключений	Продемонстрирована работа методов filter(), exclude(), get().	2
Ответы на контрольные вопросы	Полные, аргументированные ответы на все вопросы	1
Оформление отчёта	Четкая структура, наличие скриншотов, описание проделанной работы	1
ИТОГО		10

Шкала перевода баллов в оценку

- 9–10 баллов → «отлично»
- 7–8 баллов → «хорошо»
- 5–6 баллов → «удовлетворительно»
- Менее 5 баллов → «неудовлетворительно»